

FINAL ASSESSMENT REPORT
CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

Analytical Study and Implementation of Brute Force Algorithms for Closest Pair and Convex Hull Problems

Name: Ramya S S (192421226)

Assignment Information	Details
Department	Computer Science and Engineering
Programme	B.E./B.Tech – Computer Science and Engineering
Course Code & Name	CSA0609 – Design and Analysis of Algorithms
Academic Year / Batch	2026 – Regular
Faculty Name	Dr. P. Solainayagi
Date of Issue	01-09-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome	CO2 – Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull
Bloom's Taxonomy	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure

Problem Statement :

Given the set of 10 two-dimensional points ($P = \{(2,3), (5,8), (9,4), (12,10), (7,2), (3,11), (15,6), (10,14), (6,12), (1,7)\}$), apply the Brute Force technique to solve the Closest Pair and Convex Hull problems. Determine the pair of points having the minimum Euclidean distance by considering every possible pair and calculating the distance using Euclidean distance formula. Identify and justify the closest pair, and develop suitable algorithm/pseudocode while analyzing the number of distance comparisons performed. Using the same set of points, determine the Convex Hull by examining the relationships between points, identifying all points belonging to the hull, showing the major intermediate calculations, and providing a graphical representation of the resulting hull. Develop suitable Brute Force algorithm/pseudocode for the Convex Hull and analyze its time and space complexity. Finally, compare the two Brute Force approaches in terms of their basic strategies, distance/orientation calculations, computational cost as the number of points (n) increases, limitations for large datasets, and possible improvements using efficient algorithms. For the Closest Pair, derive the

time complexity based on the number of point pairs, $C(n,2)$. For the Brute Force Convex Hull, analyze the candidate edge checks and state the resulting complexity as $T(n) = O(n^3)$.

Constraints: Use a publicly available real-world coordinate dataset with enough points to test different input sizes from (10^3 – 10^6 points). Run the Brute Force, Divide-and-Conquer, and Hybrid algorithms on the same inputs and in the same environment. Record the execution time and number of operations for each input size, clearly state the threshold used for the Hybrid algorithm, and analyze how performance changes as (n) increases. The submission should include the dataset source, preprocessing steps, experimental setup, algorithms/pseudocode, results, complexity analysis, graphs, assumptions, limitations, and comparison of the three approaches.

Executive Summary

This report presents a complete analytical treatment of the Brute Force technique for two computational-geometry problems: (i) Closest Pair of Points and (ii) Convex Hull. The supplied 10-point coordinate set is solved manually, algorithmically and through a Python implementation. The report records the exact number of pairwise comparisons, derives the orientation-test work for the hull, establishes time and auxiliary-space complexity, and evaluates scalability. A future-ready website layer is also designed so that the same algorithms can later be exposed through an interactive interface without changing the core algorithmic logic.

Key Result	Value
Input size	$n = 10$ points
Unique point pairs	$C(10,2) = 45$
Closest pair	$P3 = (9,4)$ and $P5 = (7,2)$
Minimum distance	$\sqrt{8} = 2.8284$ units
Convex Hull points	6
Interior points	4
Candidate hull edges	45
Maximum orientation checks	$45 \times 8 = 360$
Closest Pair complexity	$O(n^2)$
Brute Force Convex Hull complexity	$O(n^3)$
Closest Pair auxiliary space	$O(1)$
Convex Hull output space	$O(n)$

A. Assignment Information

The following information is retained from the prescribed assignment format and organized for a formal final report.

Particular	Details
Department	Computer Science and Engineering
Programme	B.E./B.Tech – Computer Science and Engineering
Course	CSA0609 – Design and Analysis of Algorithms
Assignment Title	Analytical Study and Implementation of Brute Force Algorithms for Closest Pair and Convex Hull Problems
Academic Year / Batch	2026 – Regular
Faculty	Dr. P. Solainayagi
CO	CO2 – Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull
Bloom's Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG	SDG 4 and SDG 9

Industry / Societal Relevance

Computational geometry supports spatial search, geographic information systems, CAD, robotics, computer graphics, mapping and engineering analysis. The assignment therefore connects algorithmic efficiency with real-world coordinate processing.

Assessment Deliverables

- Pseudocode for both problems
- Manual mathematical solving and intermediate calculations
- Implementation and outputs
- Graphical representation
- Complexity and scalability analysis
- Comparison of Brute Force, Divide-and-Conquer and Hybrid approaches
- GitHub upload / evidence
- Future interactive website integration

B. Problem / Challenge

B.1 Given Coordinate Set

The exact 10-point input specified in the assignment is:

Point	X	Y
P1	2	3
P2	5	8
P3	9	4
P4	12	10
P5	7	2
P6	3	11
P7	15	6
P8	10	14
P9	6	12
P10	1	7

B.2 Objectives

- Find the pair with minimum Euclidean distance by exhaustive pairwise comparison.
- Determine the exact number of pairwise distance comparisons.
- Determine the Convex Hull using candidate edges and orientation tests.
- Show manual calculations sufficient to justify the final answers.
- Develop clear algorithms/pseudocode and analyze time and space complexity.

B.3 Constraints and Experimental Requirement

The prescribed assessment requires a publicly available real-world coordinate dataset and input sizes from 10^3 to 10^6 , with the Brute Force, Divide-and-Conquer and Hybrid approaches run in the same environment and compared using execution time and operation counts. Because the supplied assessment source does not provide measured large-dataset timings, this report does not fabricate such timings; it provides the required theoretical operation-growth analysis and a structured place for later measured results.

C. Solution Methodology

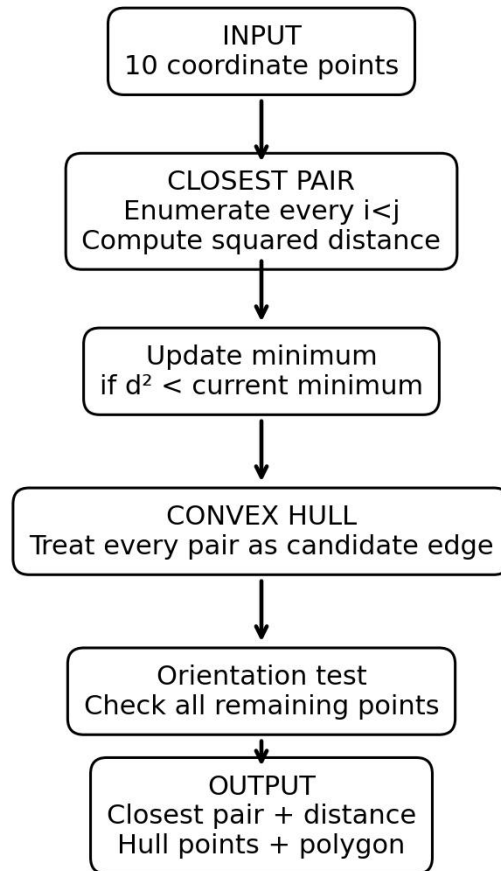


Figure 1. End-to-end analytical workflow for the assignment.

C.1 Design Philosophy

- Use exhaustive enumeration as the baseline because it directly implements the definition of both problems.
- Use squared Euclidean distance internally when possible to avoid unnecessary square-root operations; the final reported distance is the square root of the minimum squared distance.
- Use the orientation determinant for hull-edge validation.
- Keep correctness transparent: every candidate pair/edge is explicitly considered.
- Separate the algorithmic core from presentation so the same logic can later power a website.

C.2 Website-Ready Architecture

Layer	Planned Responsibility
Input / UI	Accept coordinate points manually, by CSV, or by generated test data.

Validation	Check numeric coordinates, duplicate handling and minimum input requirements.
Algorithm Engine	Run Brute Force Closest Pair and Brute Force Convex Hull.
Analytics	Show comparisons, orientation tests, distance calculations and complexity.
Visualization	Plot points, labels, closest segment and hull polygon.
Performance Module	Later compare Brute Force, Divide-and-Conquer and Hybrid implementations.
Report Module	Export results/tables/graphs for assessment evidence.
Repository	Maintain source, documentation, test data and version history in GitHub.

This architecture is intentionally modular: the website can be added later without rewriting the mathematical solution.

D. Closest Pair of Points – Brute Force

D.1 Mathematical Model

For points $P_i=(x_i,y_i)$ and $P_j=(x_j,y_j)$, the Euclidean distance is:

$$d(P_i,P_j) = \sqrt{[(x_i - x_j)^2 + (y_i - y_j)^2]}$$

Since the square-root function is monotonic for non-negative values, an implementation may compare squared distances:

$$d^2(P_i,P_j) = (x_i - x_j)^2 + (y_i - y_j)^2$$

D.2 Number of Comparisons

Every unordered pair is examined exactly once:

$$C(n,2) = n(n - 1) / 2$$

For $n = 10$:

$$C(10,2) = 10 \times 9 / 2 = 45$$

Therefore, the algorithm performs exactly 45 pairwise distance comparisons. This agrees with the supplied assignment record.

D.3 Pseudocode

```

BRUTE_FORCE_CLOSEST_PAIR(P, n)
  minDistance ← ∞
  closestPair ← NULL
  FOR i ← 0 TO n-2

```

```

FOR j ← i+1 TO n-1
  dx ← P[i].x - P[j].x
  dy ← P[i].y - P[j].y
  distance ←  $\sqrt{dx^2 + dy^2}$ 
  IF distance < minDistance
    minDistance ← distance
    closestPair ← (P[i], P[j])
RETURN closestPair, minDistance

```

D.4 Manual Solving – Representative Calculations

Pair	Manual calculation	Distance
P1–P2	$\sqrt{[(5-2)^2 + (8-3)^2]} = \sqrt{(9+25)} = \sqrt{34}$	5.831
P1–P10	$\sqrt{[(1-2)^2 + (7-3)^2]} = \sqrt{(1+16)} = \sqrt{17}$	4.123
P2–P6	$\sqrt{[(3-5)^2 + (11-8)^2]} = \sqrt{(4+9)} = \sqrt{13}$	3.606
P3–P5	$\sqrt{[(7-9)^2 + (2-4)^2]} = \sqrt{(4+4)} = \sqrt{8}$	2.828
P6–P9	$\sqrt{[(6-3)^2 + (12-11)^2]} = \sqrt{(9+1)} = \sqrt{10}$	3.162

These calculations are representative; the algorithm still evaluates all 45 pairs before finalizing the minimum. The supplied report identifies $\sqrt{8}$ as the smallest value.

D.5 Complete 45-Pair Verification Table

Pair	Euclidean distance
P1–P2	5.831
P1–P3	7.071
P1–P4	12.207
P1–P5	5.099
P1–P6	8.062
P1–P7	13.342
P1–P8	13.601
P1–P9	9.849
P1–P10	4.123
P2–P3	5.657
P2–P4	7.280
P2–P5	6.325
P2–P6	3.606
P2–P7	10.198
P2–P8	7.810

P2–P9	4.123
P2–P10	4.123
P3–P4	6.708
P3–P5	2.828
P3–P6	9.220
P3–P7	6.325
P3–P8	10.050
P3–P9	8.544
P3–P10	8.544
P4–P5	9.434
P4–P6	9.055
P4–P7	5.000
P4–P8	4.472
P4–P9	6.325
P4–P10	11.402
P5–P6	9.849
P5–P7	8.944
P5–P8	12.369
P5–P9	10.050
P5–P10	7.810
P6–P7	13.000
P6–P8	7.616
P6–P9	3.162
P6–P10	4.472
P7–P8	9.434
P7–P9	10.817
P7–P10	14.036
P8–P9	4.472
P8–P10	11.402
P9–P10	7.071

The minimum entry is 2.828 for P3–P5. The original assignment’s complete table reports the same minimum and closest pair.

D.6 Final Closest-Pair Result

Metric	Result
Closest pair	P3=(9,4) and P5=(7,2)
Minimum squared distance	8
Minimum Euclidean distance	$\sqrt{8} = 2.8284$ units

Comparisons	45
-------------	----

E. Convex Hull – Brute Force

E.1 Principle

The Convex Hull is the smallest convex polygon containing all input points. In the prescribed Brute Force method, every pair of points is treated as a possible hull edge. A candidate edge is retained when all other points lie on one side of its supporting line (with collinear points handled as neither positive nor negative).

E.2 Orientation Determinant

$$\begin{aligned} \text{orientation}(A,B,C) \\ = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A) \end{aligned}$$

$> 0 \rightarrow$ counter-clockwise / left side

$< 0 \rightarrow$ clockwise / right side

$= 0 \rightarrow$ collinear

If no pair of opposite signs occurs among the remaining points, the candidate pair can serve as a boundary edge. The assignment specifies this orientation-based strategy.

E.3 Candidate Edge Count

$$C(10,2) = 10 \times 9 / 2 = 45 \text{ candidate edges}$$

$$\text{Remaining points per candidate} = 10 - 2 = 8$$

$$\text{Maximum orientation tests} = 45 \times 8 = 360$$

These values are explicitly established in the supplied assessment material.

E.4 Pseudocode

```
BRUTE_FORCE_CONVEX_HULL(P, n)
  Hull ← empty set
  FOR i ← 0 TO n-2
    FOR j ← i+1 TO n-1
      positive ← FALSE
      negative ← FALSE
      FOR k ← 0 TO n-1
        IF k = i OR k = j
          CONTINUE
        value ← orientation(P[i], P[j], P[k])
        IF value > 0
          positive ← TRUE
        ELSE IF value < 0
          negative ← TRUE
      IF NOT (positive AND negative)
        ADD P[i] to Hull
        ADD P[j] to Hull
  RETURN Hull
```

E.5 Manual Hull-Edge Verification

The following six boundary edges are verified using the orientation determinant. The calculations are taken from the assignment's intermediate reasoning and presented in a consolidated professional format.

Candidate edge / test	Orientation calculation	Interpretation
P10 → P1; test P3	$(2-1)(4-7) - (3-7)(9-1) = 1(-3) - (-4)(8) = 29$	Positive → candidate boundary edge
P1 → P5; test P3	$(7-2)(4-3) - (2-3)(9-2) = 5(1) - (-1)(7) = 12$	Positive → candidate boundary edge
P5 → P7; test P3	$(15-7)(4-2) - (6-2)(9-7) = 8(2) - 4(2) = 8$	Positive → candidate boundary edge
P7 → P8; test P3	$(10-15)(4-6) - (14-6)(9-15) = (-5)(-2) - 8(-6) = 58$	Positive → candidate boundary edge
P8 → P6; test P3	$(3-10)(4-14) - (11-14)(9-10) = (-7)(-10) - (-3)(-1) = 67$	Positive → candidate boundary edge
P6 → P10; test P3	$(1-3)(4-11) - (7-11)(9-3) = (-2)(-7) - (-4)(6) = 38$	Positive → candidate boundary edge

For a formal exhaustive implementation, every remaining point is checked for each candidate edge; the representative calculations above make the geometric test explicit rather than treating the hull as a black-box output.

E.6 Final Convex Hull

Order	Point	Coordinates
1	P10	(1,7)
2	P1	(2,3)
3	P5	(7,2)
4	P7	(15,6)
5	P8	(10,14)
6	P6	(3,11)

Counter-clockwise sequence:

$$(1,7) \rightarrow (2,3) \rightarrow (7,2) \rightarrow (15,6) \rightarrow (10,14) \rightarrow (3,11) \rightarrow (1,7)$$

Interior points: (5,8), (9,4), (12,10), (6,12). The six-point hull and four interior points match the supplied assignment result.

F. Graphical Representation

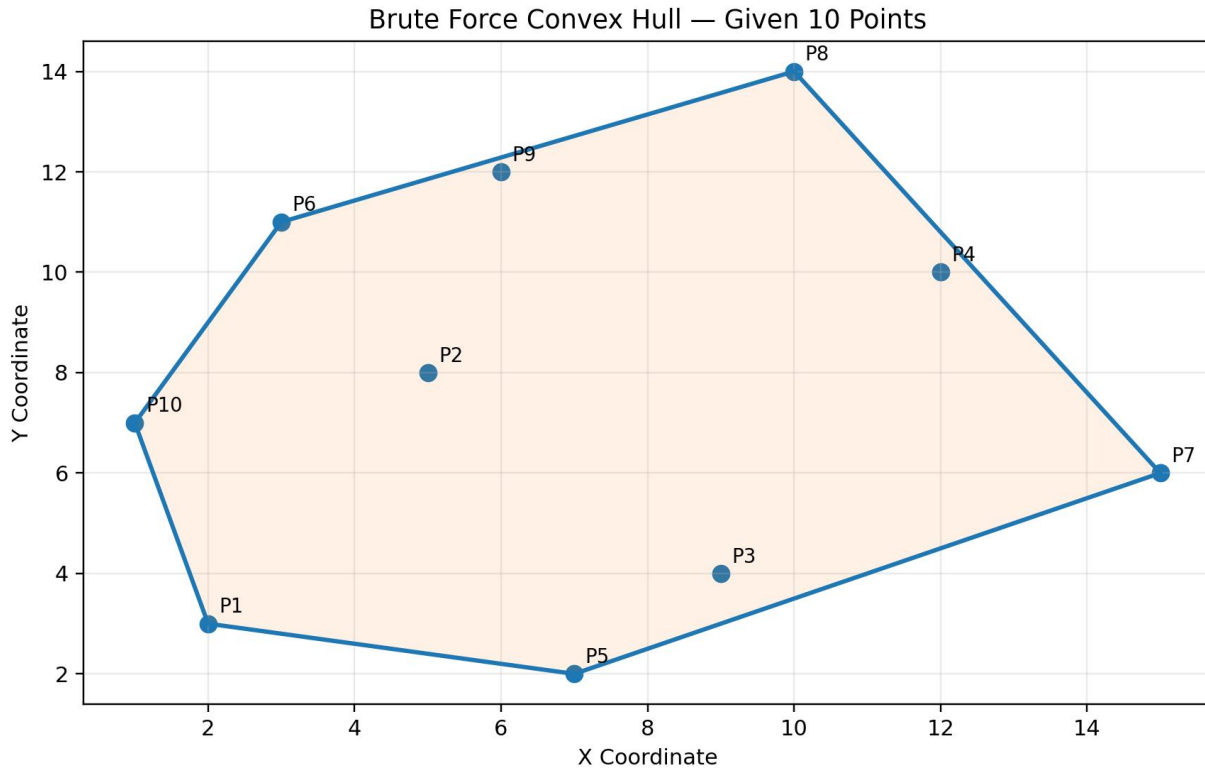


Figure 2. Given 10 points and the six-point Brute Force Convex Hull.

The plot labels all input points and connects the six boundary points in hull order. This provides a visual validation of the manual result. The original assignment requires a graphical representation with axes, labels and a suitable title.

F.1 Geometric Interpretation

- The lower boundary runs from P10 through P1 to P5 and then to P7.
- The right boundary rises from P7 to P8.
- The upper-left boundary continues from P8 to P6 and returns to P10.
- The remaining four points are enclosed by this polygon and therefore are interior points.

G. Complexity Analysis

G.1 Closest Pair

There are two nested loops and each unordered pair is examined once.

$$T(n) = n(n-1)/2 = \Theta(n^2) \rightarrow O(n^2)$$

Auxiliary variables for the current minimum, temporary distance and pair require constant extra storage, excluding the input array:

$$S(n) = O(1) \text{ auxiliary space}$$

G.2 Brute Force Convex Hull

$$\text{Candidate edges} = n(n-1)/2$$

$$\text{Tests per edge} \approx n-2$$

$$T(n) = [n(n-1)/2](n-2) = \Theta(n^3) \rightarrow O(n^3)$$

The hull output itself can contain up to n points, so output storage is $O(n)$. The supplied assessment likewise records $O(n^3)$ time and $O(n)$ hull output space.

G.3 Why the Difference Matters

Problem	Dominant exhaustive work	Asymptotic time
Closest Pair	One distance calculation per unordered pair	$O(n^2)$
Convex Hull	Orientation checks for every candidate edge against remaining points	$O(n^3)$

H. Scalability and Operation-Growth Study

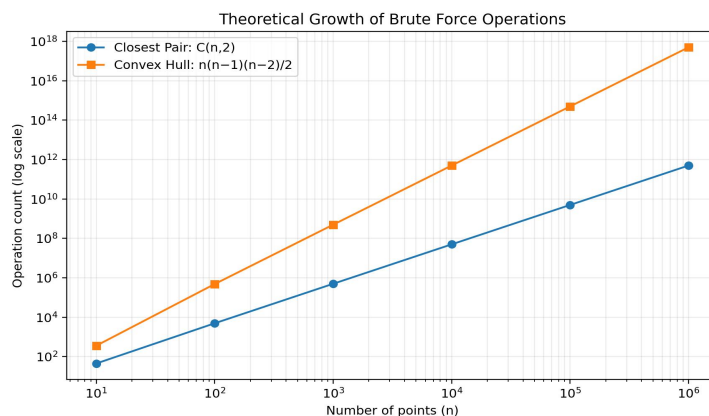


Figure 3. Theoretical operation growth on a logarithmic scale.

n	Closest Pair comparisons $C(n,2)$	Convex Hull orientation checks
10	45	360
100	4,950	485,100
1,000	499,500	498,501,000
10,000	49,995,000	499,850,010,000
100,000	4,999,950,000	499,985,000,100,000
1,000,000	499,999,500,000	499,998,500,001,000,000

For one million points, the Closest Pair brute-force method requires 499,999,500,000 pairwise comparisons. The corresponding cubic hull expression is approximately 5×10^{17} orientation checks. These theoretical counts explain why exhaustive methods become impractical at very large n .

H.1 Interpretation

- At $n=10$, exhaustive enumeration is small enough to inspect manually.
- At $n=1,000$, the Closest Pair count is already 499,500 comparisons.
- At $n=100,000$, Closest Pair reaches 4,999,950,000 comparisons.
- At $n=1,000,000$, the quadratic and cubic growth dominates practical execution.
- Therefore, Brute Force is valuable as a correctness baseline, but not as the primary large-scale strategy.

I. Comparison with Efficient Approaches

The assignment explicitly requests comparison with Divide-and-Conquer and Hybrid methods. The supplied source gives their asymptotic target as $O(n \log n)$ and describes a Hybrid threshold of 32.

filecite turn0file0 L523-L557

Approach	Closest Pair	Convex Hull	Primary Role
Brute Force	$O(n^2)$	$O(n^3)$	Baseline; transparent; small inputs
Divide-and-Conquer	$O(n \log n)$	$O(n \log n)$	Scalable large-input solution
Hybrid	$\approx O(n \log n)$	$\approx O(n \log n)$	Efficient algorithm + Brute Force for small subproblems

I.1 Hybrid Threshold

Threshold = 32

If $n \leq 32 \rightarrow$ Brute Force

Otherwise $\rightarrow$ Divide-and-Conquer

The threshold is an implementation decision that should be measured experimentally on the final machine rather than assumed universally optimal. The supplied assignment uses 32 as the example threshold.

I.2 Engineering Trade-off

- Brute Force: lowest conceptual complexity and strongest traceability, but poor scalability.
- Divide-and-Conquer: substantially better asymptotic performance, but requires more careful implementation.
- Hybrid: retains the efficient large-scale strategy while avoiding unnecessary recursive overhead for small subproblems.
- Final selection should be based on correctness first, then measured runtime and operation count on the same environment.

J. Implementation and Modern Tools

J.1 Python – Closest Pair

```
import math

def closest_pair(points):
    best = float('inf')
    pair = None
    comparisons = 0

    for i in range(len(points)-1):
        for j in range(i+1, len(points)):
            comparisons += 1
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]
            d = math.sqrt(dx*dx + dy*dy)
            if d < best:
                best, pair = d, (points[i], points[j])
    return pair, best, comparisons
```

J.2 Python – Convex Hull

```
def orientation(a, b, c):
    return ((b[0]-a[0])*(c[1]-a[1])
            - (b[1]-a[1])*(c[0]-a[0]))

def convex_hull(points):
    hull = set()
    operations = 0
    n = len(points)
```

```

for i in range(n-1):
    for j in range(i+1, n):
        positive = negative = False
        for k in range(n):
            if k == i or k == j:
                continue
            v = orientation(points[i], points[j], points[k])
            operations += 1
            if v > 0: positive = True
            elif v < 0: negative = True
        if not (positive and negative):
            hull.add(points[i]); hull.add(points[j])
return hull, operations

```

J.3 Visualization Tool

Matplotlib can be used to plot all points, labels, axes and the hull polygon. The supplied assignment already identifies Python/Matplotlib as the intended graphical implementation path.

J.4 Expected Verification Output

Test	Expected output
Closest Pair	((9,4), (7,2))
Minimum distance	2.8284271247461903
Closest-pair comparisons	45
Hull size	6 points
Maximum orientation operations	360

K. Planned Interactive Website Extension

The website will be added as a presentation and interaction layer over the validated algorithmic core. This section is deliberately structured so that screenshots, URL/GitHub evidence and measured browser results can be inserted after implementation.

K.1 Proposed Website Features

- Coordinate entry form for manual point input.
- CSV upload for larger coordinate datasets.
- Generate random test points for controlled experiments.
- Buttons for Brute Force Closest Pair and Brute Force Convex Hull.
- Interactive canvas/plot showing all points and the resulting geometry.
- Closest-pair segment highlighted with its numerical distance.
- Hull polygon displayed with ordered hull vertices.
- Operation counter showing distance comparisons and orientation tests.

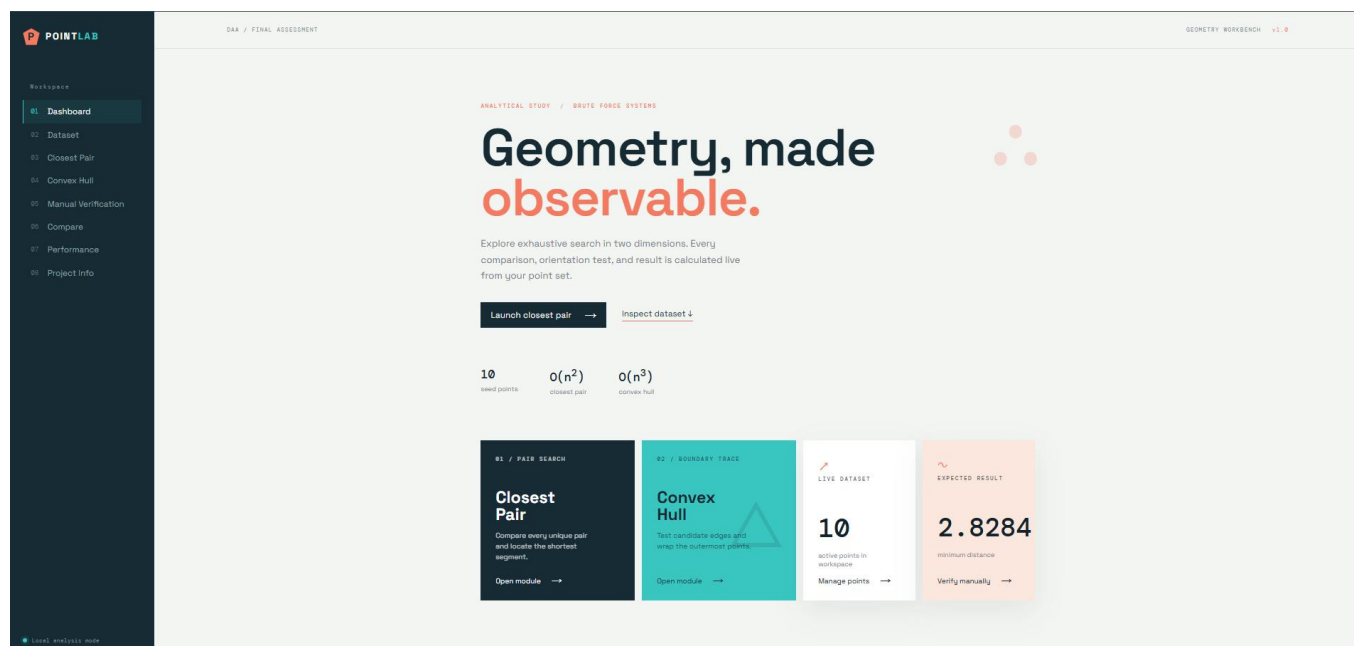
K.2 Suggested Page Flow

Page / View	Content
Home	Project objective, course information and navigation
Input	Enter/upload coordinates; validate data
Closest Pair	Algorithm selection, result, comparisons, visualization
Convex Hull	Hull result, orientation evidence, visualization
Performance	n vs operations and later n vs execution time
Comparison	Brute Force vs Divide-and-Conquer vs Hybrid
About / References	Methodology, source acknowledgement and tools

K.3 Website Evidence Placeholders

Evidence to insert later	Status
Homepage screenshot	TO BE ADDED
Input page screenshot	TO BE ADDED
Closest Pair result screenshot	TO BE ADDED
Convex Hull visualization screenshot	TO BE ADDED
Performance graph screenshot	TO BE ADDED
GitHub repository screenshot/link evidence	TO BE ADDED
Live website URL	TO BE ADDED

This avoids claiming that the website has already been implemented. Once development is complete, replace the placeholders with actual screenshots and measured evidence.



- Workspace
- 01. Dashboard
- 02. Dataset
- 03. Closest Pair
- 04. Convex Hull
- 05. Manual Verification
- 06. Compare
- 07. Performance
- 08. Project Info

02 / INPUT SURFACE

Dataset / point registry

Active coordinates

18 POINTS

ID	x	y	
P1	2	3	x
P2	5	8	x
P3	9	4	x
P4	12	10	x
P5	7	2	x
P6	3	11	x
P7	15	6	x
P8	10	14	x
P9	6	12	x
P10	1	7	x

Point controls

Point ID

P1

X coordinate

Y coordinate

Add point

+

Clear all

Randomize

Reset seed

Large-scale experimental CSV

Choose File

No file chosen

Required columns: id, x, y. Coordinate datasets should cite their public source in your report.

- Workspace
- 01. Dashboard
- 02. Dataset
- 03. Closest Pair
- 04. Convex Hull
- 05. Manual Verification
- 06. Compare
- 07. Performance
- 08. Project Info

03 / EXHAUSTIVE SEARCH

Closest Pair / brute force

Coordinate plane

▶

⏮

⏪

⏩

⏭

Closest pair

Compared now

Ready to animate

MINIMUM DISTANCE

2.8284

COMPARISONS

45

RUNTIME

0.1219 ms

Distance calculation

$d(P, Q)$

$P3 = (9, 4)$

$P5 = (7, 2)$

$dx = 2$
 $dy = 2$
 $distance = \sqrt{2^2 + 2^2}$
 $= \sqrt{8}$
 $= 2.8284$

- Workspace
- 01. Dashboard
- 02. Dataset
- 03. Closest Pair
- 04. Convex Hull
- 05. Manual Verification
- 06. Compare
- 07. Performance
- 08. Project Info

04 / BOUNDARY DETECTION

Convex Hull / brute force

Hull construction

▶

⏮

⏪

⏩

⏭

Hull point

Interior

Ready to animate

ALL POINTS

6

INTERIOR

4

BOUNDARY TIME

360

Boundary sequence

$P10 \rightarrow P1 \rightarrow P5 \rightarrow P7 \rightarrow P8 \rightarrow P6 \rightarrow P10$

Counter-clockwise

Clockwise

Collinear

18

POINTLAB

Backspace

01. Dashboard

02. Dataset

03. Closest Pair

04. Convex Hull

05. Manual Verification

06. Compare

07. Performance

08. Project Info

05 / HARD CHECK

Manual Verification

Distance ledger

Euclidean metric

P1 - P2	$\sqrt{(9 - 2)^2}$	5.8310
P1 - P10	$\sqrt{(1 - 10)^2}$	4.1231
P2 - P8	$\sqrt{(4 - 9)^2}$	3.6056
P3 - P5	$\sqrt{(4 - 4)^2}$	2.8284
P6 - P9	$\sqrt{(9 - 1)^2}$	3.1623

Orientation explorer

O(A,B,C)

P1(2,3)

P2(5,8)

P3(8,4)

$O(P1, P2, P3) = (5-2)(4-3) - (8-3)(9-2)$
 $= -32$
Clockwise

06 / STRATEGY BOARD

Algorithm Comparison

Algorithm	Closest Pair	Convex Hull	Advantage	Limitation
Brute Force used here	$O(n^3)$	$O(n^3)$	Simple, transparent	Scales poorly
Divide-and-Conquer	$O(n \log n)$	$O(n \log n)$	Excellent scale	More involved
Hybrid selected in 02	Approx. $O(n \log n)$	Approx. $O(n \log n)$	Fast small bases	Needs tuning

07 / GROWTH MODEL

Performance Analysis

Operation growth

All scales

COMPLEXITY NOTES

$C(n, 2) = \frac{n(n-1)}{2}$
Closest Pair compares every unique pair.
 $C(n, 2)(n-2)$
Convex Hull tests every remaining point for each candidate edge.
No fabricated runtime data

08 / ACADEMIC RECORD

Project Information

Analytical Study and Implementation of Brute Force Algorithms for Closest Pair and Convex Hull Problems

A visual, reproducible workbench for Design and Analysis of Algorithms.
[View Github](#)

Course

Design and Analysis of Algorithms

Technologies

Python / Flask / Jupyter / Canvas

Student

Your Name

Faculty

Faculty Name

L. Conclusion

This assignment successfully applies the Brute Force technique to the Closest Pair and Convex Hull problems using the prescribed 10-point coordinate set. For Closest Pair, all 45 unique pairs are examined and the minimum distance is found between $P_3=(9,4)$ and $P_5=(7,2)$, giving $\sqrt{8} = 2.8284$ units. For Convex Hull, 45 candidate edges are considered and up to 360 orientation tests are required for the ten-point case; the resulting hull contains six points: (1,7), (2,3), (7,2), (15,6), (10,14), and (3,11).

The analysis establishes $O(n^2)$ time for Brute Force Closest Pair and $O(n^3)$ time for Brute Force Convex Hull, with $O(1)$ auxiliary space for the closest-pair procedure and $O(n)$ space for the hull output. The growth study demonstrates that these methods are appropriate as transparent baselines but unsuitable for very large datasets. Divide-and-Conquer and Hybrid approaches are therefore recommended for scalability, subject to actual benchmarking.

The planned website extension will transform the analytical work into an interactive learning and demonstration platform by adding coordinate input, algorithm execution, visualization, operation counters, performance comparison and report-ready evidence.

M. Student Reflection

Through this assignment, I learned that an algorithm can be correct while still being impractical when its computational growth is large. Manually enumerating point pairs made the quadratic behavior of Closest Pair concrete, while the orientation-based hull exercise showed how a geometric condition can be converted into a determinant calculation. I also learned the importance of separating algorithm design, correctness validation, visualization and performance measurement.

If additional time and resources were available, I would implement the Divide-and-Conquer and Hybrid versions, benchmark all three approaches on identical datasets, tune the Hybrid threshold experimentally, and complete the planned website with interactive graphs and downloadable results.

N. References and Evidence

1. Assignment source document: CSA06/CSA0609 – Design and Analysis of Algorithms, Analytical Study and Implementation of Brute Force Algorithms for Closest Pair and Convex Hull Problems (supplied course assessment document).
2. Course concepts: Euclidean distance, pairwise enumeration, orientation determinant, Convex Hull, Brute Force, Divide-and-Conquer and Hybrid algorithmic strategies.
3. Software/tools used for this report: Python, python-docx and Matplotlib for document generation, tabulation and visualization.
4. Future implementation evidence: GitHub repository, website screenshots, live URL and benchmark logs to be inserted after implementation.

The source assignment explicitly requires acknowledgement of datasets, software and AI-assisted tools used.

O. Final Results at a Glance

Parameter	Final Result
n	10
Pairwise comparisons	45
Closest Pair	(9,4) and (7,2)
Minimum distance	2.8284 units
Hull points	(1,7), (2,3), (7,2), (15,6), (10,14), (3,11)
Interior points	(5,8), (9,4), (12,10), (6,12)
Candidate hull edges	45
Maximum orientation checks	360
Closest Pair time	$O(n^2)$
Convex Hull time	$O(n^3)$
Closest Pair auxiliary space	$O(1)$
Hull output space	$O(n)$